

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Introduction
- ▶ Difference between .Net core and .Net Framework
- ▶ .Net Project Structure
- ▶ Nuget Package Manager
- ▶ Routing in ASP.Net Core
- ▶ Common Status Code And HTTP Verbs

Let's Start

.Net core - Introduction

- ▶ **What is the ASP.NET Core?**
- ▶ ASP.NET Core is not an upgraded version of ASP.NET.
- ▶ ASP.NET Core is completely rewriting that work with the .net Core framework. It is much faster, configurable, modular, scalable, extensible, and has cross-platform support.
- ▶ It can work with both .NET Core and .net framework via the .NET standard framework. It is best suitable for developing cloud-based such as web applications, mobile applications, and IoT applications.
- ▶ ASP.NET Core was primarily designed to make the most important part of the ASP.NET components under the concept learn and the compose framework where the previous ASP.NET components were released under a variety of different licenses periodically, The ASP.NET Core framework is a completely open-source framework.
- ▶ Apart from the other parts of the framework of the .NET framework libraries, the ASP.NET Core is primarily designed from scratch to be the platform-agnostic that performs seamlessly. It will allow the ASP.NET Core apps to be deployed on the various platforms or the o/s such as the macOS or Linux-based servers or certain devices.

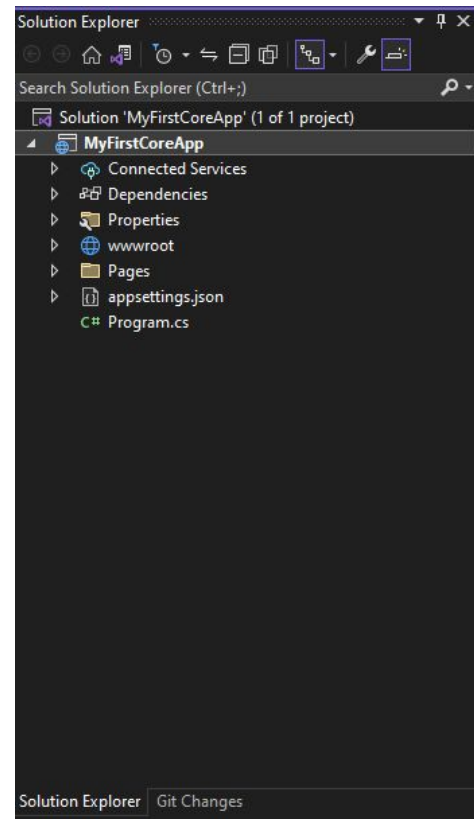
.Net core - Introduction

► What are the differences between .NET Core and .NET Framework?

.NET Core	.NET Framework
Completely open-source.	Few components are open-source.
Compatible with Linux, Windows, and Mac operating systems.	Compatible with only Windows.
Does not support desktop application development.	Supports web and desktop application development.
Supports microservices development.	Supports web and desktop application development.
Lightweight for Command Line Interface (CLI).	Heavy for Command Line Interface.
Is flexible in installation, i.e. you can install components on a needed basis.	Must be installed as a whole package

.Net core - Project Structure

- ▶ Here, we will learn about the project structure and significance of each file created by ASP.NET Core 6.0 application template using Visual Studio 2022. The following is a default project structure when you create an empty ASP.NET Core application in Visual Studio.
- ▶ **Note:** ASP.NET Core project files and folders are synchronized with physical files and folders. If you add a new file or folder in project folder then it will directly reflect in the solution explorer. You don't need to add it in the project explicitly by right clicking on the project.
- ▶ **.csproj**
ASP.NET Core 6.0 no longer uses .xproj and project.json files. Instead, it fully utilizes the .csproj file to manage the project.
- ▶ With the new version, you can edit the .csproj settings directly in Visual Studio or any text editor. To do so, right-click on the project in Visual Studio and select "Edit Project File" to modify the .csproj content.



.Net core - Project Structure

- ▶ This change was introduced in ASP.NET Core 2.0, and it carries forward into ASP.NET Core 6.0. The .csproj file now contains all the necessary project configuration and references, making it more streamlined and familiar for developers working with .NET projects.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net6.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

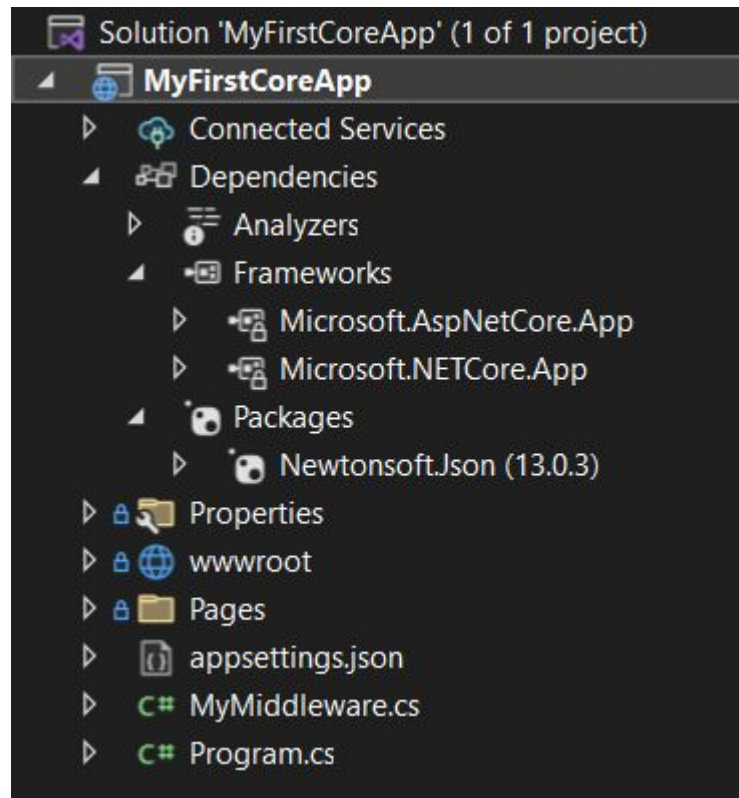
  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
  </ItemGroup>

</Project>
```

- ▶ The .csproj for the above project looks like below.
- ▶ The .csproj file includes settings related to targeted .NET Frameworks, project folders, NuGet package references etc.

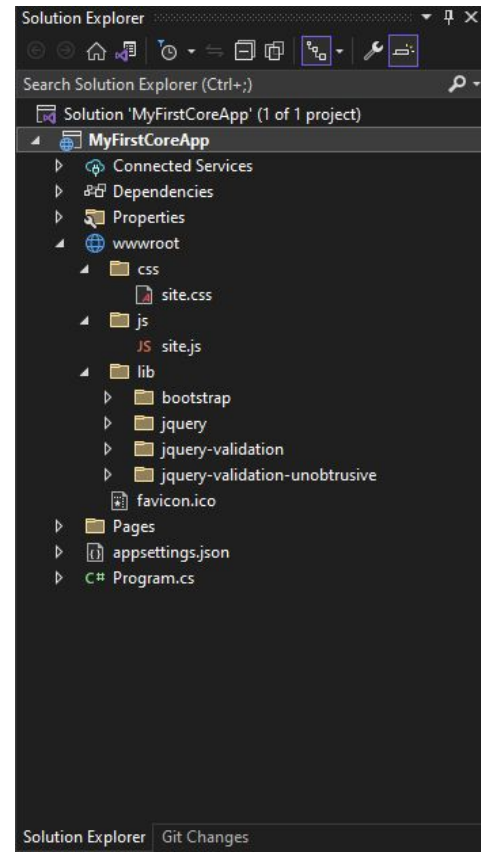
NuGet Package Manager

- ▶ **Dependencies:** The Dependencies in the ASP.NET Core 6.0 project contain all the installed server-side NuGet packages, as shown below.
- ▶ Right click on "Dependencies" and then click "Manage NuGet Packages.." to see the installed NuGet packages
- ▶ You can install all other required server side dependencies as NuGet packages from Manage NuGet Packages window or using Package Manager Console.



.Net core - Project Structure

- ▶ **ASP.NET Core - wwwroot Folder**
- ▶ By default, the wwwroot folder in the ASP.NET Core project is treated as a web root folder. Static files can be stored in any folder under the web root and accessed with a relative path to that root.
- ▶ In the standard ASP.NET application, static files can be served from the root folder of an application or any other folder under it. This has been changed in ASP.NET Core. Now, only those files that are in the web root - wwwroot folder can be served over an http request. All other files are blocked and cannot be served by default.
- ▶ Generally, there should be separate folders for the different types of static files such as JavaScript, CSS, Images, library scripts etc. in the wwwroot folder as shown in the given screenshot.



Routing in ASP.NET Core

- ▶ **What is Routing?**

- ▶ Mechanism to map incoming HTTP requests to the appropriate controller actions.

- ▶ **Types of Routing:**

- ▶ Conventional Routing
 - ▶ Attribute Routing

Routing in ASP.NET Core

▶ Conventional Routing:

- ▶ Defined in Startup.cs using `app.UseEndpoints`.
- ▶ Follows a predefined pattern (e.g., `{controller=Home}/{action=Index}/{id?}`).

▶ Attribute Routing:

- ▶ Defined directly above controllers or actions using `[Route]` attribute.
- ▶ More flexible and explicit.

Routing in ASP.NET Core

► Conventional Routing:

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
});
```

► Attribute Routing:

```
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    [HttpGet("{id:int}")]
    public IActionResult GetOrder(int id) { ... }
}
```

Common Status Code And HTTP Verbs

▶ Common Status Codes:

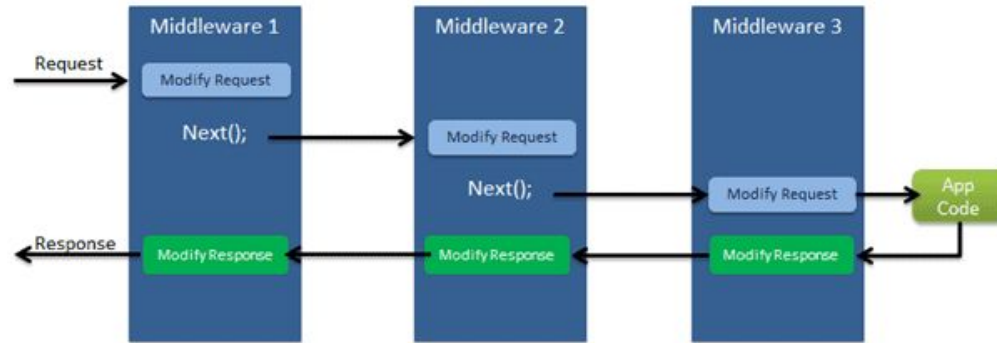
- ▶ 200 OK – Success
- ▶ 400 Bad Request – Invalid input
- ▶ 404 Not Found – Resource not found
- ▶ 500 Internal Server Error – Server error

▶ Common HTTP Verbs:

- ▶ GET – Retrieve data
- ▶ POST – Create data
- ▶ PUT – Update data
- ▶ DELETE – Delete data

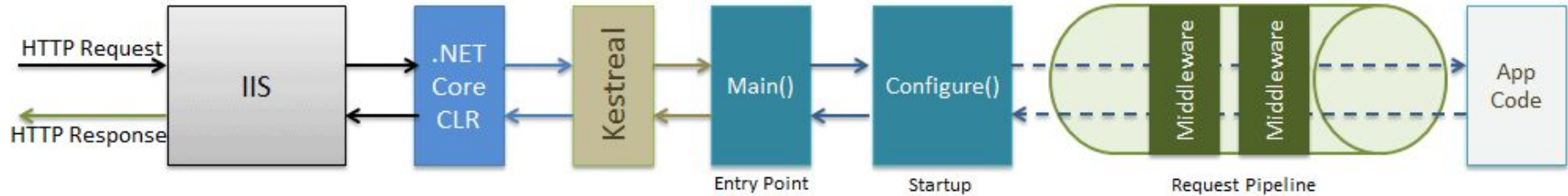
.Net core - Middleware

- ▶ **ASP.NET Core - Middleware:** ASP.NET Core introduced a new concept called Middleware. A middleware is nothing but a component (class) which is executed on every request in ASP.NET Core application. In the classic ASP.NET, HttpHandlers and HttpModules were part of request pipeline. Middleware is similar to HttpHandlers and HttpModules where both needs to be configured and executed in each request.



- ▶ Typically, there will be multiple middleware in ASP.NET Core web application. It can be either framework provided middleware, added via NuGet or your own custom middleware. We can set the order of middleware execution in the request pipeline. Each middleware adds or modifies http request and optionally passes control to the next middleware component. The following figure illustrates the execution of middleware components.
- ▶ Middlewares build the request pipeline. The following figure illustrates the ASP.NET Core request processing.

.Net core - Middleware



- ▶ **Configure Middleware:** We can configure middleware in the `Program.cs` file using `WebApplication` instance. The following example adds a single middleware using `Run` method which returns a string "Hello world from 1st middleware" on each request.
- ▶ In the given example, `Run()` is an extension method on `WebApplication` instance which adds a terminal middleware to the application's request pipeline. The above configured middleware returns a response with a string "Hello world from 1st middleware" for each request.

```
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 1st middleware");  
});
```

.Net core - Middleware

- ▶ Mostly there will be multiple middleware components in ASP.NET Core application which will be executed sequentially. The Run method adds a terminal middleware so it cannot call next middleware as it would be the last middleware in a sequence. The following will always execute the first Run method and will never reach the second Run method.
- ▶ To configure multiple middleware, use `Use()` extension method. It is similar to `Run()` method except that it includes next parameter to invoke next middleware in the sequence.

```
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 1st middleware");  
});  
  
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 2nd middleware");  
});
```

- ▶ **Add Built-in Middleware Via NuGet:** ASP.NET Core is a modular framework. We can add server side features we need in our application by installing different plug-ins via NuGet. There are many middleware plug-ins available which can be used in our application.

.Net core - Dependency Injection

- ▶ **ASP.NET Core - Dependency Injection:** ASP.NET Core is designed from scratch to support Dependency Injection. ASP.NET Core injects objects of dependency classes through constructor or method by using built-in IoC container.
- ▶ **Built-in IoC Container**
- ▶ ASP.NET Core framework contains simple out-of-the-box IoC container which does not have as many features as other third party IoC containers. If you want more features such as auto-registration, scanning, interceptors, or decorators then you may replace built-in IoC container with a third party container.
- ▶ The built-in container is represented by `IServiceProvider` implementation that supports constructor injection by default. The types (classes) managed by built-in IoC container are called services.
- ▶ **There are basically two types of services in ASP.NET Core:**
 1. Framework Services: Services which are a part of ASP.NET Core framework such as `IApplicationBuilder`, `IHostingEnvironment`, `ILoggerFactory` etc.
 2. Application Services: The services (custom types or classes) which you as a programmer create for your application.
- ▶ In order to let the IoC container automatically inject our application services, we first need to register them with IoC container.

.Net core - Dependency Injection

- ▶ **Registering Application Service:** Consider the following example of simple `ILog` interface and its implementation class. We will see how to register it with built-in IoC container and use it in our application.

```
public interface ILog
{
    void info(string str);
}

class MyConsoleLogger : ILog
{
    public void info(string str)
    {
        Console.WriteLine(str);
    }
}
```

- ▶ ASP.NET Core allows us to register our application services with IoC container, in the `Program.cs` there is a instance of `WebApplicationBuilder` and inside that there is `Service` property of type `IServiceCollection`, which is used to register application services.

.Net core - Dependency Injection

- ▶ Let's register `ILog` with IoC container in Program.cs as shown below.

```
5 // Add services to the container.
6 builder.Services.AddRazorPages();
7 builder.Services.AddSingleton<ILog, MyConsoleLogger>();
8
9 var app = builder.Build();
10
11 // Configure the HTTP request pipeline.
12 if (!app.Environment.IsDevelopment())
13 {
14     app.UseExceptionHandler("/Error");
15     // The default HSTS value is 30 days. You may want to change this for production scenarios
16     app.UseHsts();
17 }
18
19 app.UseMyMiddleware();
20 app.UseHttpsRedirection();
21 app.UseStaticFiles();
22 app.UseRouting();
23 app.UseAuthorization();
24 app.MapRazorPages();
25 app.Run();
```

.Net core - Dependency Injection

- ▶ **Understanding Service Lifetime:** Built-in IoC container manages the lifetime of a registered service type. It automatically disposes a service instance based on the specified lifetime.
- ▶ The built-in IoC container supports three kinds of lifetimes:
 1. **Singleton:** IoC container will create and share a single instance of a service throughout the application's lifetime.
 2. **Transient:** The IoC container will create a new instance of the specified service type every time you ask for it.
 3. **Scoped:** IoC container will create an instance of the specified service type once per request and will be shared in a single request.
- ▶ **Constructor Injection:** Once we register a service, the IoC container automatically performs constructor injection if a service type is included as a parameter in a constructor.

.Net core - Dependency Injection

- ▶ For example, we can use `ILog` service type in any MVC controller. Consider the following example.

```
public class HomeController : Controller
{
    ILogger _log;

    public HomeController(ILogger log)
    {
        _log = log;
    }
    public IActionResult Index()
    {
        _log.info("Executing /home/index");

        return View();
    }
}
```

- ▶ **Action Method Injection:** In the example given on the right side, there might be situations where we only require the dependency service type within a specific action method. To achieve this, we can utilize the `[FromServices]` attribute along with the service type parameter in the method signature. This allows us to access the required service directly within that particular action method.

- ▶ In the provided example on the left, when working with an IoC container, the constructor of `HomeController` will be automatically provided with an instance of `MyConsoleLogger`. No additional manual intervention is required. Furthermore, the IoC container will handle the creation and disposal of an instance of `ILog` as per its registered lifetime.

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public HomeController()
    {
    }

    public IActionResult Index([FromServices] ILogger log)
    {
        log.info("Index method executing");

        return View();
    }
}
```

.Net core - Dependency Injection

- ▶ **Property Injection:** Built-in IoC container does not support property injection. You will have to use third party IoC container.
- ▶ **Get Services Manually:** It is not required to include dependency services in the constructor. We can access dependent services configured with built-in IoC container manually using `RequestServices` property of `HttpContext` as shown below.

```
public class HomeController : Controller
{
    public HomeController()
    {
    }
    public IActionResult Index()
    {
        var services = this.HttpContext.RequestServices;
        var log = (ILog)services.GetService(typeof(ILog));

        log.info("Index method executing");

        return View();
    }
}
```

- ▶ It is recommended to use constructor injection instead of getting it using `RequestServices`.

.Net core - Dependency Injection

- ▶ **Property Injection:** Built-in IoC container does not support property injection. You will have to use third party IoC container.
- ▶ **Get Services Manually:** It is not required to include dependency services in the constructor. We can access dependent services configured with built-in IoC container manually using `RequestServices` property of `HttpContext` as shown below.

```
public class HomeController : Controller
{
    public HomeController()
    {
    }
    public IActionResult Index()
    {
        var services = this.HttpContext.RequestServices;
        var log = (ILog)services.GetService(typeof(ILog));

        log.info("Index method executing");

        return View();
    }
}
```

- ▶ It is recommended to use constructor injection instead of getting it using `RequestServices`.

Thank you